SARBLOH: Structured Geometric Perception and Speculative Code Execution for Autonomous ARC-AGI-3 Agents

Harsidak Singh Banwait
Independent Researcher
harsidak.singh.banwait.13@gmail.com

Abstract

ARC-AGI-3 measures fluid intelligence through interactive games that an agent has never seen, scored by how few actions it needs compared with a human. In such unknown environments, perceiving the game state correctly is the first thing an agent needs. Yet when frontier models are shown raw 64×64 integer grids, they spend hundreds of actions rediscovering mechanics that a human grasps at a glance. We present SARBLOH AGENT, an agent built around a deterministic host-side perception engine and a code-as-action runtime. In $\approx$35 ms per frame, the engine produces three aligned views: an object-centric state with identity tracking across steps, a briefing that separates verifiable geometric invariants from affordance hypotheses paired with one-action tests, and an annotated spatial map. The agent writes Python world models and action plans over these views, supported by an append-only context that reuses 98.4% of the prefix cache and a priority scheduler that shares one GPU across 25 concurrent games. In a perception lab of eight presentations, the structural facts the engine computes carried over exactly to 23 of 23 unseen games, while role guesses made from a single frame did not, which is why the engine states the first as facts and the second as guesses with a test. On one RTX PRO 6000, SARBLOH AGENT clears 143 of 183 public levels in 8.2 hours and scores 16.53 on the hidden Kaggle evaluation, up from 10.98 in the preceding build.

Code: https://github.com/Harsidak/SARBLOH

1 Introduction

Human visual cognition exhibits remarkable inductive fluidity when dealing with novel interactive environments. Observation of an unfamiliar task for less than a hundred milliseconds allows humans to absorb fundamental physical and operational elements. Identifying agent embodiment, understanding spatial boundaries, recognizing temporal and resource gauges, and deducing final goal affordances can occur without the need for explicit complex manuals (Spelke and Kinzler 2007). While these abstractions are inductive and subject to revision, they provide the necessary structural scaffolding that enables humans to solve complex tasks.

The ARC-AGI-3 benchmark (ARC Prize Foundation 2026a) was established to evaluate this core facet of artificial general intelligence: the efficiency of novel skill acquisition under strict epistemic uncertainty (Chollet 2019). Unlike static visual reasoning benchmarks, ARC-AGI-3 requires an agent to operate in a sequential environment that includes an undisclosed series of compositional levels. In this kind of environment, the agent receives no natural language directives, action semantic descriptions, or termination conditions. Furthermore, the benchmark evaluates performance through Relative Human Action Efficiency (RHAE), where the earned score scales quadratically with the ratio of human baseline actions to agent actions. Consequently, each exploratory or misdirected action incurred by an agent penalizes its final score quadratically, rendering brute-force trial-and-error catastrophically inefficient.

When multimodal models are deployed in these environments, their primary failure mode is visual grounding and spatial representation. When an environment frame is serialized into an unparsed matrix of discrete integer tokens, a state-of-the-art (SOTA) reasoning model has to use tens of thousands of tokens to reconstruct the dynamics of these environments—such as connectivity, containment, and coordinate alignment. In preliminary controlled investigations (Section 3), exposing a frontier model to raw 64×64 integer grids resulted in over 200–300 exploratory actions on the navigation task, leading to exhaustion of the action budget and task failure. In stark contrast, providing the same model with a visually aligned rendering and structural guidance enabled direct task resolution in 6 to 12 actions. Geometric briefing enabled direct task resolution in 6 to 12 actions. On interactive abstract reasoning benchmarks, perceptual structuring is the primary operational bottleneck.

This visual bottleneck binds with acute severity under the rigorous constraints of the offline Kaggle ARC Prize track. In this setting, candidate agents are evaluated on undisclosed, held-out game environments without external network connectivity. Computation is strictly bounded to a single NVIDIA RTX PRO 6000 GPU (96 GB VRAM) for a total execution window of nine hours across all evaluated games, and all model weights must be open-source. Under these stringent computational and temporal boundaries, an agent cannot afford iterative self-correction or excessive reflective generation over poorly formatted observations; it

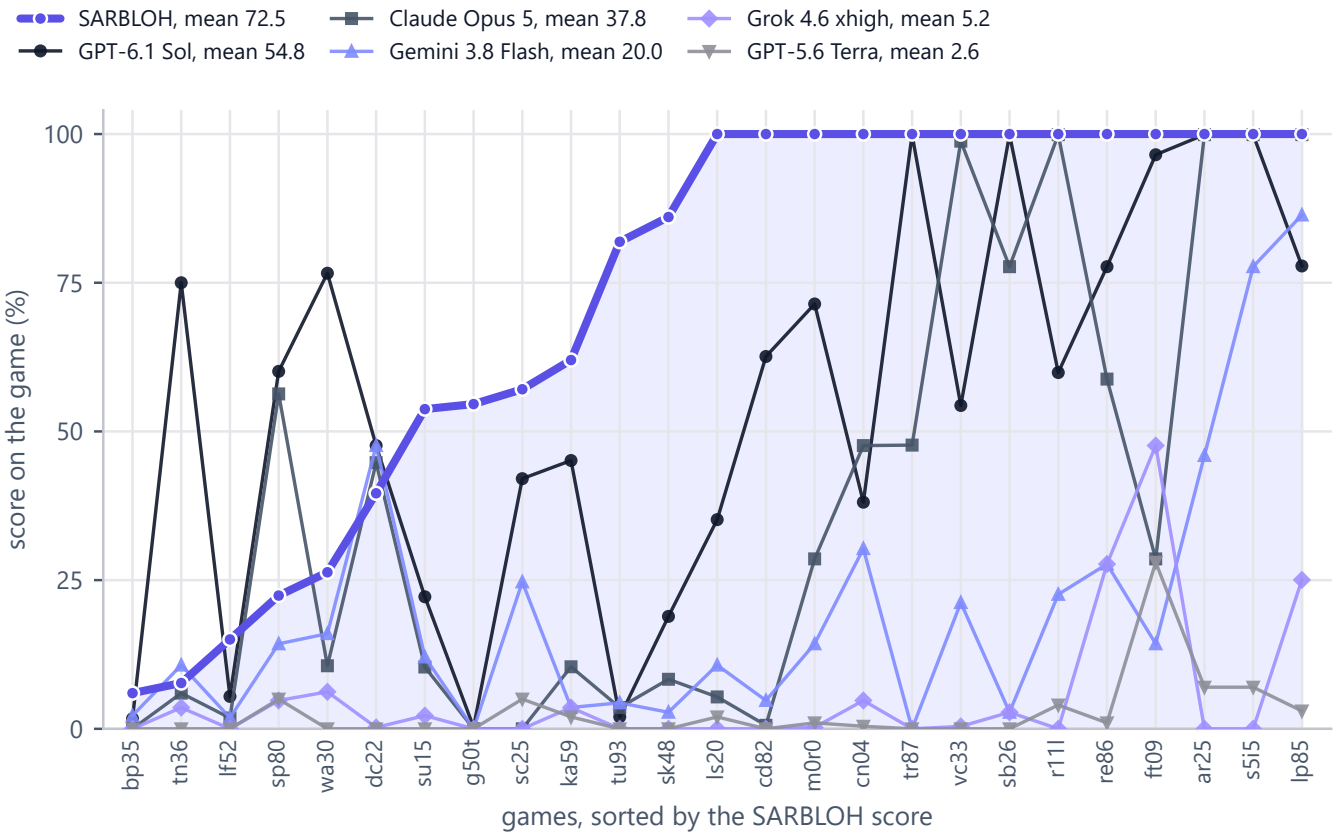


Figure 1 | Score on the 25 public ARC-AGI-3 games. One point per game, sorted by the SARBLOH score: the thick indigo line is SARBLOH with an open-weight Qwen3.8-Flash-Next model on one GPU, the other five lines are frontier models. The SARBLOH line takes, per game, the best of our runs (8 to 18 per game); our best single run has a mean of 59.4. The mean over the 25 games is 72.5 for SARBLOH, 54.8 for GPT-6.1 Sol, 37.8 for Claude Opus 5, 20.0 for Gemini 3.8 Flash, 5.2 for Grok 4.6 xhigh and 2.6 for GPT-5.6 Terra. SARBLOH reaches 100% on 13 games. These are public-game scores, which are not comparable to the hidden-set score in Table 1.

requires high-fidelity, deterministic perceptual abstractions that minimize token latency and preserve action efficiency.

In this work, we present SARBLOH AGENT, an autonomous open-weight agent architecture designed specifically for resource-constrained, high-efficiency interactive reasoning on ARC-AGI-3. At the core of SARBLOH AGENT is a zero-parameter, host-side *perception engine* that operates in $\mathcal{O}(HW)$ algorithmic time (≈35 ms per frame), transforming each raw grid state into three structurally synchronized modalities: (i) an object-centric symbolic representation with stable bipartite identity tracking across temporal transitions, (ii) a geometric briefing that isolates mathematically proven spatial invariants (BFS lattice reachability, topological containment, midpoint tethers, and dihedral symmetries under $D_4 \times \mathbb{Z}^+$) from affordance hypotheses paired with unit verification actions, and (iii) an affordance-annotated visual map.

Surrounding this perceptual foundation, SARBLOH AGENT integrates an execution-grounded planning engine operating exclusively via a persistent Python REPL. Rather than issuing myopic primitive actions, the agent synthesizes programmatic world models, explores trajectory trees via algorithmic graph search, and commits multi-step speculative execution paths through a unified primitive that continuously monitors environmental invariants and halts upon sensory prediction discrepancies. To maximize inference throughput on our single-GPU budget, we serve Qwen3.8-Flash-Next—a 125B parameter Mixture-of-Experts architecture activating 6B parameters per token—quantized via AutoRound W4A16 and served using SGLang with EAGLE speculative decoding. By enforcing a strictly monotonic, append-only context protocol, the runtime achieves a 98.4% prompt prefix-cache hit rate across hybrid linear-recurrent and attention layers, while a utility-driven priority scheduler dynamically multiplexes the GPU across 25 concurrent games based on expected marginal RHAE gains.

Our main contribution is the perception engine. Prior ARC-AGI-3 agents hand the model the frame as integer arrays or ASCII grid, a rendered image, or both. We instead compute those structural facts exactly on the host, with no learned parameters, and give the model identity-tracked objects, proven geometric invariants and falsifiable affordance hypotheses in place of pixels. The controlled study in Sec-

tion 3 shows that this change of representation, rather than a stronger planner, accounts for most of the saving in actions, and in our build history the engine, together with refined context guardrails, takes the official hidden-set score from 10.98 to 16.53, a 51% relative gain on top of an already-tuned system (Section 5). For reference, the Milestone #1 winner scored 1.21, and our first Prime Agent port scored between 0.00 and 0.27.

2 Preliminaries and Related Work

2.1 Environment

An ARC-AGI-3 environment (ARC Prize Foundation 2026a) is an uninstructed, discrete-time game made of an ordered sequence of levels $\{1, \ldots, n\}$. At each step t the agent sees a frame $g_t \in \{0, \ldots, 15\}^{H \times W}$ with $H, W \leq 64$, the episode state $s_t \in \{\texttt{NOT_FINISHED}, \texttt{WIN}, \texttt{GAME_OVER}\}$ and the number of levels cleared. It then picks an action from

$$\mathcal{A} = \{\texttt{ACTION1}, \ldots, \texttt{ACTION5}, \texttt{ACTION6}(x, y), \texttt{ACTION7}, \texttt{RESET}\}, \tag{1}$$

where ACTION1–ACTION4 are directional, ACTION5 and ACTION7 are game-specific, and ACTION6(x, y) is a click at pixel (x, y). Nothing is explained up front: what the actions do, which object the agent controls and what wins the level must all be inferred from transitions. Early levels introduce single mechanics and later levels compose them (Spelke and Kinzler 2007).

Games are scored by *Relative Human Action Efficiency* (RHAE). Let h_l be the human baseline for level l (the upper-median human action count) and a_l the agent's action count. Then

$$S_l = \min\left(1.15, \left(\frac{h_l}{a_l}\right)^2\right), \qquad E = \frac{\min\left(\sum_{l=1}^{k} l,\ \sum_{l=1}^{n} l\, S_l\right)}{\sum_{l=1}^{n} l}, \tag{2}$$

where $k \leq n$ is the number of levels cleared, and the benchmark score is the mean of E over games. Three consequences drive our design. Extra actions are penalized quadratically, so an agent that needs twice the human count keeps only 25% of the credit. Thinking and host-side computation are free, and only actions sent to the environment count in a_l, which favours planning in code over trial and error. And the weight l pushes the score toward later levels.

Offline evaluation and resource bounds. The official Kaggle track runs a submitted notebook on held-out games with no network access. It gets one NVIDIA RTX PRO 6000 (Blackwell, 96 GB VRAM), 176 GB of host RAM and a nine-hour wall-clock limit for the whole suite, and all weights must be open and packaged with the notebook. The budget is therefore decode throughput, since reasoning is bounded by tokens per second, and the length of prompt that can stay in the warm prefix cache without eviction or recomputation.

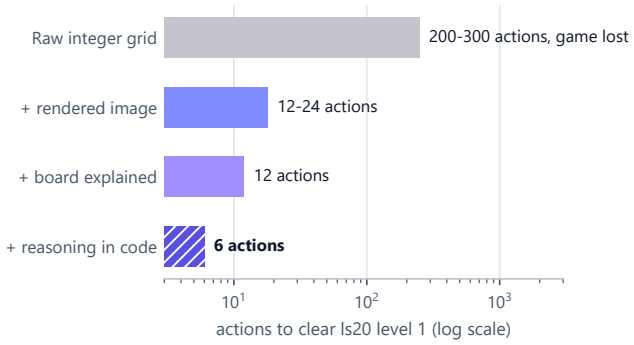


Figure 2 | Controlled ablation of input observation modalities on the first level of ls20, evaluated using an identical frontier reasoning model. Serializing the environment as raw integer tokens induces catastrophic exploration failure (200–300 actions, exceeding budget). Providing a rendered visual frame reduces required actions by over an order of magnitude (14× speedup), while augmenting the state with deterministic geometric relations and programmatic execution achieves super-human efficiency (6 actions versus human baseline ≈10).

2.2 Related Work

How Prior Agents Perceive the Game. Prior ARC-AGI-3 systems (Table 1) differ in planning, memory and tooling, but they share one perception design: the frame reaches the model as raw data, and the model recovers structure itself. The Duck harness, passes composite ASCII grids together with visual renderings of the board (Tufa Labs 2026; ARC Prize Foundation 2026b). The second- and third-place Kaggle systems feed annotated frames to Gemma-4-31B, which emits JSON action sequences directly (ARC Prize Foundation 2026b). *VISTA* (Han et al. 2026) adds a multimodal visual harness that keeps every frame losslessly. Arcgentica (Knutsen and Klein 2026) and Prime Agent (Karten et al. 2026) expose the game through a persistent Python REPL, so the model must write its own code to parse the grid.

In every case the observation is an integer or ASCII grid, a rendered image, or both. Objects, connectivity, containment and symmetry are left to the model to infer from pixels or characters, which is the step our motivating study (Section 3) finds most costly. We know of no prior ARC-AGI-3 agent that computes these facts exactly on the host, tracks object identity across frames, or attaches a one-action test to each uncertain hypothesis.

3 Motivation

Before building the agent, we ran controlled ablations to find where a language model wastes its action budget in interactive reasoning.

The Representation Ladder. We evaluated a frontier model (Gemini 3.1 Pro) on the first level of ls20. The model, prompt and environment were held fixed, and only the form of the observation changed (Figure 2).

Given the raw 64×64 integer matrix, the model spent

	Hidden Kaggle set (offline, 1 GPU, 9 h)			Public set (frontier APIs, unbounded)				
	Sarbloh this work	**Duck** Milestone #1, 1st	**Reki** Milestone #1, 2nd	**Prime Agent** open harness	**Arcgentica** Symbolica	**AVO** NVIDIA	**Provider Adapters** GPT-5.5	**Provider Adapters** Gemini 3.8 Flash
Backbone	Qwen3.8-Flash-Next (MoE)	Qwen 3.6 27B	Gemma-4-31B	Claude Opus 5	Claude Opus 4.6	Opus 5 / GPT-5.6 Sol	GPT-5.5	Gemini 3.8 Flash
Open weights	✓	✓	✓	–	–	–	–	–
Score	**16.53**	**1.21**	**0.87**	**95.5**	**36.1**†	**100.0**†	**99.99**†	**≈99.9**†

Table 1 | ARC-AGI-3 systems under two evaluation protocols. The left group runs offline on the hidden Kaggle games with open weights on one GPU; the right group calls frontier APIs on the public games. The two scores are not directly comparable. †Reported by the system's authors on the public games and not certified by the organizers; the Provider Adapters scores are as reported to us, and the Gemini figure is approximate. No adapter result for Claude is available.

200 to 300 actions colliding with invisible boundaries and exhausted the budget without reaching the goal. Reading reachability off 4,096 serialized integers is a task attention heads do unreliably (Rahmanzadehgervi et al. 2024). A rendered image of the same board cut this to 12–24 actions, and adding a structural decomposition (terrain, gauges, color parity) cut it to 12. Letting the model write search scripts in a sandbox finished the level in 6. We observed that most of the gain came from the representation of the environment, not from more Chain-of-Thought.

The Oracle Briefing. To separate planning failures from perception failures, we gave the model an advanced `ls20` level with screenshots alone, and with screenshots plus an "oracle briefing" stating the action semantics, goal, mechanical dependencies and on-screen entities. Without the briefing the model burned its whole allocation on inert artifacts. With it, the model cleared the level in 6 actions, better than the author's blind playthrough (≈10). The planner was capable all along; it was held back by inaccurate state induction.

The Failure of Sparse Deltas. Sending only the cells that changed saves tokens but causes perceptual myopia. Actions often have non-local effects: stepping on a trigger tile in `ls20` changes an icon in a distant status panel. With only local updates the agent misses such coupling and concludes its action did nothing, so we keep whole-frame representations.

Design Principle. Oracle briefings do not exist for held-out games, so the agent must build its own abstractions. ARC-AGI-3 rests on human core knowledge (Spelke and Kinzler 2007), which is geometric and topological (connectivity, enclosure, reachability, symmetry). Operational semantics, such as which object is the agent or what wins the level, are latent and can only be learned by acting. The agent should therefore be told the first kind of fact outright and only offered the second kind as guesses to test, the design rule stated in Section 4.1.

Summary. (i) With the model and prompt fixed, changing only the observation took the first level of `ls20` from 200–300 actions to 6. (ii) Given an oracle briefing, the same model cleared an advanced level in 6 actions, so the planner was not the bottleneck; perception was. (iii) Sparse deltas hide non-local effects, so the agent sees whole frames.

4 Methodology

Our first agent, a port of Prime Agent (Karten et al. 2026), scored 0.00 to 0.27 on the hidden set and restarted levels more than 350 times in ten games without once finding a goal. The model could write code. It could not tell what it was looking at. This section rebuilds the agent around one rule for what the model is told about a frame (Section 4.1), the engine that applies it (Section 4.2), what broke when that engine met live play (Section 4.3), the interface through which the agent acts (Section 4.4), the serving loop that keeps all of it inside one GPU (Section 4.5) and the designs we tried and dropped (Section 4.6). Figure 3 shows one real turn of the finished system.

4.1 The Measured–Guessed Rule

We did not start from the rule; a perception lab of nine presentations, graded against an answer key written before any of them ran, produced it (Appendix A, Figure 4).

A text-only encoder that states every cell flat scored 3 of 28. In the beggining our first perception module printed the fitted lattice, a symbol per cell, the palette, connected components and object tracking. Everything it said was correct but, most of the heavy lifting was done by the model itself as the model has to identify what each cell represents. It named no agent, no goal and no gauge, and it marked the move counter of `r11l` as screen decoration.

Role guesses with tests raised the score to 20, and fusing them with computed relations raised it to 25. The Role Ledger scored every object against eight game roles and printed each top guess with its evidence and the cheapest action that would confirm or refute it (20/28, about 380

(a) map before
level 1, sent with (b)



(b) briefing, engine output (excerpt)

```
BRIEFING (arrow keys)
MEASURED (certain):
- Floor is a 5x5 tile grid.
- obj 4 (blue 6-cell shape): sits inside a black box that is NOT plain floor;
    the floor reaches the box's bottom side in 5 steps (the only way in)
- obj 6 (white/light grey plus): reachable in 6 steps
  ...
GUESSES (check with one action):
- obj 10 (blue/orange square) = YOU (80%) - test: press one arrow key: it should
    move one tile and the gauge should drop
- obj 15 (yellow bar) = MOVES LEFT (75%) - test: take any action and check it
    shrinks by one unit
  ...
```

(c) the agent's ipython cell at turn 7, its first action in the level

```
r = await play(["1"])
print(r.lines)
print(r.stopped, r.surprise)
new = r.obs
d = frame_diff(obs.board, new.board)
print(d)
```

(d) cell output (excerpt)

```
play: 1 of 1 moves done
#0 UP: 50 cells changed at r40-49 c34-38; the HUD changed (2 cells)
frame_diff: 0 (10 cells) moved by [-5, 0]
frame_diff: b (15 cells) moved by [-5, 0]
[state after step #0] level 1 of 7 (0 done) | 1 actions spent (1 this level),
    1999 left
```

(e) map after
obj 10 one tile up



Figure 3 | One real turn, end to end (`ls20`, level 1, run of 27 October; transcript and frames as logged). (a) The map the engine drew for the opening frame. (b) An excerpt of the briefing sent with it: measured facts first, then guesses, each with a one-action test. (c) The agent's first action in the level, written as Python at turn 7 after six turns spent only reading `observe()`: one key press, the test that the briefing proposed for YOU. (d) What the kernel printed: the player (orange and blue, 25 cells) moved one tile of 5 cells up and the HUD lost two cells. (e) The map after the move. One action confirmed two guesses, obj 10 = YOU and obj 15 = MOVES LEFT.

tokens). The Fused Briefing added the relations that code computes exactly—reachability, enclosure, midpoint tethers, shape equivalence—as a separate tier above the guesses (25/28, about 370 tokens). It scored 8.3× the Legacy Encoder with 89% fewer tokens.

Held-out games separated the two kinds of statement. We subsequently evaluated cross-distribution generalization across 23 held-out official game environments excluded during engine prototyping. the grid reconstruction was able to achieve 100% cell accuracy across all games, and morphological segmentation reliably preserved entity boundaries and spatial tracking without degradation. Whereas topological and geometric invariants hold unconditionally by mathematical construction, high-level semantic roles (such as agent embodiment or interactive affordances) inherently possess visual ambiguity prior to actuation. Consequently, the engine treats role labels not as brittle, hardcoded classifications, but as provisional inductive hypotheses paired with minimal unit-action verification probes. This decoupling ensures that any transient misclassification of initial sprite affordances is immediately resolved upon the first interaction step, while preserving complete coordinate, topological, and geometric stability across the entire scene.

The rule. *State what can be measured as fact; state what must be inferred as a guess that carries a one-action test.* We term this the **Measured–Guessed Rule**. A measured relation is computed from the grid by code with no learned parameters, so it is true by construction and the model need not check it. A role cannot be read from one frame, and a role stated as fact plants *perceptual misinduction*—a wrong belief about what an object is or does—that the model then spends actions on. A role stated as a guess with a test costs at most one action to settle, and under RHAE, where cost grows with the square of the action count, that cap is the point.

The rest of this section applies the rule five times. View 1 states object identity across frames as fact (Section 4.2). View 2 states geometric relations as fact. The role scorer states roles as guesses, each with its test (Table 2). View 3 draws both on one map and marks every guessed role with a question mark. Finally, `play()` treats the agent's own world model as a guess and checks each of its predictions against the next frame (Section 4.4).

4.2 Perception: From Pixels to Stated Facts

The engine is the Fused Briefing rewritten to work on the grid inside the agent, with host-side NumPy and Pillow and no learned parameters. On 225 real frames (the first frame and eight random actions on each of 25 offline games) it raised no errors; the briefing took a median of 42 ms (maximum 0.38 s) and the picture 40 ms. Each frame yields three views that share one object-identity space (Figure 5). Algorithm 1 gives the whole mapping from grid to views in one place; reading it shows that every learned or heuristic step (role scoring) comes after every exact one.

View 1: re-segmenting each frame renamed objects, so identity is tracked, not recomputed. The perception engine identifies the background canvas as the modal color index across the frame and extracts all remaining four-

(a) Legacy Encoder, 3/28
text only: palette, grid, objects; ~3,500 tokens

```
PALETTE
  '.' darkgray  rgb=[52, 56, 56] cells=2609 (terrain)
  '=' gray      rgb=[104, 104, 108] cells=892 (terrain)
  'K' black     rgb=[0, 4, 8] cells=439 (terrain)
  'Y' yellow    rgb=[256, 220, 8] cells=84 (object)
  'B' blue      rgb=[32, 148, 256] cells=45 (object)
  'R' red       rgb=[248, 64, 56] cells=12 (object)
  'O' orange    rgb=[256, 136, 32] cells=10 (object)
  'W' white     rgb=[256, 256, 256] cells=3 (object)
  '#' lightgray rgb=[204, 204, 208] cells=2 (object)
     000000000011111111112222222222333333333344444444455555555556666
     012345678901234567890123456789012345678901234567890123456789012
  0  KKKK.................................
  1  KKKK.................................
  2  KKKK.................................
  3  KKKK.................................
  4  KKKK.................................
  5  KKKK.................................
  6  KKKK.................................
  7  KKKK.................................
  8  KKKK.................................=======
  9  KKKK.............................=KKKKKK
 10  KKKK.............................=KKKKKK
 11  KKKK.............................=KKBBBK
 12  KKKK.............................=KKKKBK
 13  KKKK.............................=KKBKBK
 14  KKKK.............................=KKKKKK
 15  KKKK.............................=KKKKKK
 16  KKKK.................................=======
 17  KKKK.................................=====
 18  KKKK.................................=====
 19  KKKK.................................=====
 20  KKKK.................................=====
 21  KKKK.................................=====
  ... (183 lines in all)
```

(b) Role Ledger, 20/28
roles + tests; ~380 tokens



(c) Fused Briefing, 25/28
map, MEASURED facts, GUESSES; ~370 tokens



Figure 4 | Three of the nine lab presentations on the opening frame of `ls20`, with rubric score (of 28) and text cost. (a) The Legacy Encoder, text only (excerpt of 183 lines, grid rows cut at 44 columns). (b) The Role Ledger: a role map and, below it, each guess with its evidence and a one-action test. (c) The Fused Briefing, the presentation we kept: a schematic map with object tags, shape threads and tethers, then MEASURED facts and GUESSES as separate tiers. The other six presentations are in Figure 11.

connected monochromatic regions as discrete entities because standard morphological segmentation re-indexes components in raster reading order at every step, dynamic entities moving across the board would continually swap indices, causing synthesized Python scripts to silently reference incorrect objects. To maintain temporal consistency, SARBLOH anchors object identifiers on the initial frame of each level and preserves identity across transitions through a multi-stage bipartite matching algorithm. This procedure sequentially matches entities based on rigid spatial invariance, spatial translation with identical geometry, chromatic state updates with stationary coordinates, and topological overlap exceeding fifty percent for deformable structures, assigning fresh identifiers only to newly instantiated components. As a result, each identifier reliably tracks the same physical entity throughout an entire level. In addition to the tracked object table, the spatial state is serialized as an alphanumeric coordinate grid and paired with a differential change ledger detailing the bounding box and cell count modified by each transition. Furthermore, environmental modifications occurring within the three-cell peripheral screen boundary are isolated as heads-up display telemetry, preventing passive counter decrements or timer ticks from being misinterpreted as operational task progress.

View 2: a picture does not state relations, so the engine computes them. Whether an object can be reached, how many steps it takes, whether a hollow fits another object, or whether two shapes are the same up to rotation cannot be read reliably off a rendered frame (Rahmanzadehgervi et al. 2024). The engine first splits the frame into *terrain*—colours that cover more than 8% of the frame—and things (corridors, edge gauges, multi-colour assemblies joined by union-find over 8-connected components, panel icons and dotted figures). It then computes seven kinds of measured fact:

- **Reachability**: breadth-first search over the detected *lattice*—the tile grid the player moves on—gives step counts from the likely agent to every target ("`obj 6: reachable in 6 steps`").
- **Enclosure**: closed outlines around empty space, matched against objects of the same size.
- **Midpoint tethers**: an object held at the midpoint of two others by thin lines, so moving one end moves it ("`obj 25 held at midpoint of obj 16 and 36`").
- **Alignment**: shared rows or columns between objects.
- **Dihedral symmetry**: equality of two shapes under the eight rotations and mirrorings of the square and integer scaling ($D_4 \times \mathbb{Z}^+$), as in "`obj 12 (drawn 2x bigger), mirrored left-right OR turned 90 deg clockwise, becomes exactly obj 4`".
- **Copies**: identical shapes counted as a set (lives, items).
- **Lopsided colouring**: a symmetric shape with asymmetric colours, which usually marks a direction or a state.

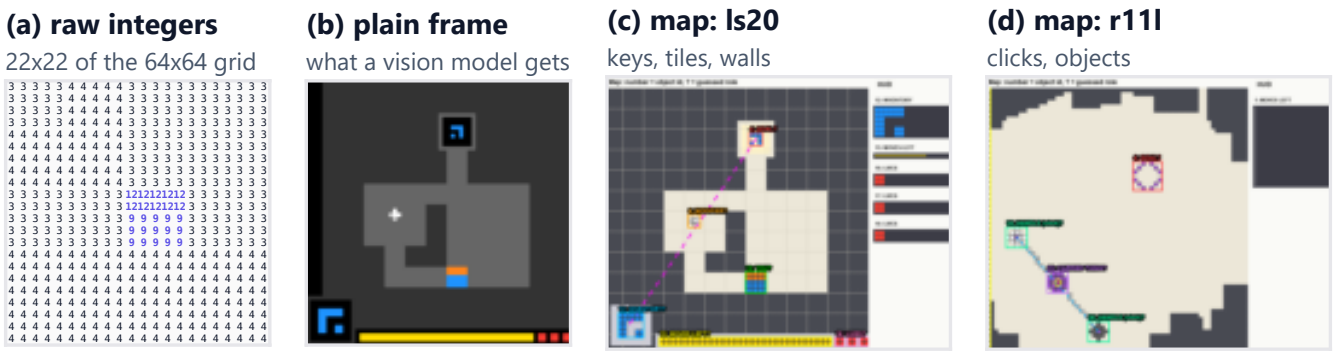


Figure 5 | One ARC-AGI-3 frame, four ways. (a) What a text-only model receives: a 22×22 window of the 64×64 integer grid, with the player's cells in colour. (b) The rendered frame. (c, d) The engine's maps for the opening frames of `ls20` (arrow keys) and `r11l` (clicks): floor in light tones, walls hatched, the HUD moved to a side panel, every object tagged with its id and guessed role (`id: ROLE?`). Magenta dashed threads join shapes equal under rotation, mirroring or scale; yellow rings mark an object held at the midpoint of two others.

Role	Evidence (scored)	One-action test
YOU	Compact, multi-coloured, unique, on the lattice	Press one arrow: it moves one tile
GOAL	Hollow box; shape repeated in the HUD	Bring the matching thing into it
MOVES LEFT	Long bar at the screen edge	Any action: it shrinks one unit
LIVES	Identical copies in the HUD	Watch it on a failure or reset
INVENTORY	HUD icon matching a world object	Watch it after the modifier
MODIFIER	Small symbol, lopsided colours	Step on it: the inventory changes
HANDLE	Tether end; on/off colour pair	Click it: something jumps
CARRIED THING	Held between two tether ends	Move a handle: it follows

Table 2 | Role guesses. Each is printed with its confidence and a test that settles it in one action. On held-out games these guesses were right about half the time or less (Table 6), which is why they are never stated as facts.

In `ls20`, the symmetry relation connects the HUD icon to the shape inside the goal enclosure, while in `r11l`, tether detection identifies the disc suspended between two handles. The perception lab confirmed that both relationships were discoverable exclusively through these geometric computations (Appendix A.3).

Roles: the engine guesses what objects are for, and says how to check. Non-terrain objects are scored against eight roles with fixed evidence points from game-design priors (Table 2); a role's confidence is its score over the object's total score plus one. Each guess is printed with its one-action test, so a wrong guess costs one action instead of a level.

The briefing shows at most 20 fact and guess lines in the deployed configuration (30 in the build of Section 4.3): up to a third for guesses, the rest shared between the fact kinds, with facts about the guessed agent and goal first. The remainder is counted ("`+N more facts`") and stays readable in code through `observe().briefing` (Section 4.3 explains why the lines are shared). Full briefings for the two lab frames are in Appendix B.

View 3: words go in the text, not in the pixels. A small vision encoder reads rendered text unreliably, and a sentence drawn into the image is a sentence the model must transcribe before it can use it. The map therefore carries only short tags (`10: YOU?`) that repeat ids from the text, while every sentence travels as text. It draws structure (floor in light tones, walls hatched, the lattice as fine lines, look-alike shapes joined by dashed threads, tethers marked with rings and the HUD magnified in a side panel (Figure 5c,d)).

Observation schedule. The full view (briefing, letter grid and map) is sent at the start of a level, after a context drain, and when the agent calls `observe()` after acting, at most once per state. After other actions the agent gets the change ledger, a letter crop around the changed cells with a 3-cell margin, and the new map. The object table stays on the host and is read in code (`observe().objects`).

The engine states facts correctly on the frames it was designed on, and its guesses are honest about being guesses. However, the engine was engineered basd on the dynamics(geometry, topology, etc.) observed in public games, and it is not a learned model. It does not generalize perfectly to unseen games or to frames from live play.

4.3 What Broke Under Live Play

The lattice test flipped between frames, so reachability facts appeared and vanished mid-level. On `ls20` the engine lost its 5×5 tile grid after one move, because the gauge in the status box added edges that lie off the grid; on `ka59` it lost its 3×3 grid whenever the player touched another object. Each flip deleted every reachability fact from the briefing, so the agent was told on one frame that the goal was 5 steps away and on the next frame nothing. We

Algorithm 1: PERCEIVE: one frame to three views

Input: Grid $g_t \in \{0..15\}^{H \times W}$, tracker T, previous lattice Λ', legal actions $\mathcal{A}$
Output: Briefing B, tracked objects O, map M, change ledger δ

$O \leftarrow \textsc{Segment}(g_t)$;
$(O, \delta) \leftarrow T.\textsc{Match}(O)$; ▷ *five passes; ids fixed per level*
$\mathcal{T} \leftarrow \{c \in \{0..15\} : |g_t{=}c| > 0.08\,HW\}$; ▷ *terrain*
$L \leftarrow \textsc{Layout}(g_t, \mathcal{T})$;
foreach *thing* $e \in L$ **do**
  $e.hud \leftarrow \textsc{AtScreenEdge}(e)$;
  $e.id \leftarrow \textsc{MajorityId}(e, O)$;
$e_{\text{you}} \leftarrow \textsc{LikelyAgent}(L, \mathcal{A})$;
$\Lambda \leftarrow \textsc{DetectLattice}(L \setminus \text{HUD})$ **or** Λ' ; ▷ *keep last lattice if it fits*
$F \leftarrow \textsc{Reach}(e_{\text{you}}, \Lambda) \cup \textsc{Enclosures}(L) \cup \textsc{Tethers}(L) \cup \textsc{Symmetries}_{D_4 \times \mathbb{Z}^+}(L)$; ▷ *measured*
foreach *thing* $e \in L$ **do**
  $\boldsymbol{\sigma}_e \leftarrow \textsc{ScoreRoles}(e, F, \mathcal{A})$;
  $\hat{\rho}_e \leftarrow \arg\max_r \boldsymbol{\sigma}_e[r]$;
  $p_e \leftarrow \boldsymbol{\sigma}_e[\hat{\rho}_e] / (1 + \sum_r \boldsymbol{\sigma}_e[r])$; ▷ *guessed*
$B \leftarrow \textsc{Facts}(F) \,\|\, \textsc{Guesses}(\hat{\boldsymbol{\rho}}, \boldsymbol{p}, \textsc{Tests}(\hat{\boldsymbol{\rho}}))$;
$M \leftarrow \textsc{DrawMap}(L, \hat{\boldsymbol{\rho}}, F)$;
return B, O, M, δ

now leave status boxes out of the lattice test and keep the level's previous lattice while it still fits the frame. On the same 225 frames this removed every flip. A fact that is true by construction is only true if its inputs are the right inputs.

Alignment facts crowded out every other kind when the briefing had a single cap. In scenes containing numerous entities, pairwise coordinate alignments expand combinatorially. For example, the initial frame of `tr87` yields 60 alignment relations compared to only 7 enclosure relations and 1 symmetry relation, while `bp35` produces 792 alignment pairs. Under an unweighted global line cap, these high-frequency alignment facts quickly saturate the briefing, crowding out sparse but vital invariants such as symmetry relations and operational role hypotheses (Figure 6a). To prevent dominant relation categories from monopolizing the observation space without discarding structural information, the briefing scheduler allocates line quotas proportionally across invariant types and reserves up to one-third of the total display for affordance hypotheses. This balanced sharing policy ensures that critical geometric symmetries and candidate roles remain visible in context, while surplus relations are summarized as aggregate counts rather than silently truncated (Figure 6b).

4.4 The Action Interface

Separate tools for code and for actions split one plan across many turns. In our early builds the agent had one tool to run Python and another to submit moves. In the traces of those builds it modelled the game in one tool, left it to send a single move with the other, and came back to re-derive its state, so one plan cost many turns and every turn cost a model call. We merged the two: the agent acts only from inside its

(a) one cap of 30 lines
23 alignment, 7 enclosure,
0 look-alike, 0 guess lines

```
obj 6 encloses empty 3x3 space
obj 8 encloses empty 3x3 space
obj 18 encloses empty 3x3 space
obj 20 encloses empty 3x3 space
obj 31 encloses empty 3x3 space
obj 33 encloses empty 3x3 space
obj 51 encloses empty 3x31 space
obj 5 left of obj 6 r5-9
obj 5 left of obj 7 r5-9
obj 5 left of obj 8 r5-9
obj 5 above obj 17 c13-17
obj 5 above obj 30 c13-17
obj 5 above obj 44 c15-17
obj 5 above obj 47 c15-17
obj 5 above obj 51 c15-17
obj 5 above obj 62 c15-17
obj 6 left of obj 7 r5-9
obj 6 left of obj 8 r5-9
obj 6 above obj 18 c23-27
obj 6 above obj 31 c23-27
obj 6 above obj 44 c23-27
obj 6 above obj 51 c23-27
obj 7 left of obj 8 r5-9
obj 7 above obj 19 c36-40
obj 7 above obj 32 c36-40
obj 7 above obj 44 c36-40
obj 7 above obj 51 c36-40
obj 8 above obj 20 c46-50
obj 8 above obj 33 c46-50
obj 17 left of obj 18 r14-18
```

(b) lines shared by kind
20 alignment, 7 enclosure,
1 look-alike, 2 guess lines

```
obj 6 encloses empty 3x3 space
obj 8 encloses empty 3x3 space
obj 18 encloses empty 3x3 space
obj 20 encloses empty 3x3 space
obj 31 encloses empty 3x3 space
obj 33 encloses empty 3x3 space
obj 51 encloses empty 3x31 space
obj 5 left of obj 6 r5-9
obj 5 left of obj 7 r5-9
obj 5 left of obj 8 r5-9
obj 5 above obj 17 c13-17
obj 5 above obj 30 c13-17
obj 5 above obj 44 c15-17
obj 5 above obj 47 c15-17
obj 5 above obj 51 c15-17
obj 5 above obj 62 c15-17
obj 6 left of obj 7 r5-9
obj 6 left of obj 8 r5-9
obj 6 above obj 18 c23-27
obj 6 above obj 31 c23-27
obj 6 above obj 44 c23-27
obj 6 above obj 51 c23-27
obj 7 left of obj 8 r5-9
obj 7 above obj 19 c36-40
obj 7 above obj 32 c36-40
obj 7 above obj 44 c36-40
obj 7 above obj 51 c36-40
obj 47 rotated = obj 62
(+40 more facts in observe())
obj 63 = MOVES LEFT (75%), test: t..
obj 5, 6, 7, 8, 17, 18, 19, 20, 30..
```

Figure 6 | The briefing lines shown for the first frame of `tr87`, both computed by the current engine from one `Briefing` object. (a) One cap of 30 lines in the engine's fact order: alignment facts (grey) push out the symmetry fact and both role guesses (hatched). (b) The policy in use: lines shared between fact kinds and up to a third for guesses; the 40 facts not shown are counted and remain readable in code. Object descriptions are shortened for print.

Python kernel, in the CodeAct style (Wang et al. 2024), with

```
r = await play(path, predict=f)
```

where `path` is up to 100 moves and `predict` is an optional world model $\hat{f}(s, a) \to \hat{s}$ written by the agent. It is the only change recorded for the build that moved the hidden-set score from 5.14 to 5.65. One cell can now model a level, search it and play the whole path (Figure 3c).

The agent's world model is a guess, so `play()` checks it instead of trusting it. Algorithm 2 defines the execution contract between the planned trajectory and the environment, specifying the exact conditions that govern path termination. An inaccurate state prediction does not interrupt execution. Instead, `play()` logs the first action whose predicted state differs from the true grid outside the HUD, records the mismatched coordinates, and continues along the trajectory. The path halts only when a level or game terminates, when consecutive identical actions collide with an obstacle without altering a single pixel, or when the revisit guard identifies a board state already visited during the current attempt. The loop never stalls to verify an operational rule, ensuring that any inaccuracies in the internal world model are resolved immediately upon receiving the next frame.

A revisit guard looked free offline and was routed around live. The *revisit guard* terminates path execution whenever the board state, excluding the peripheral three-cell HUD bor-

Algorithm 2: PLAY: running a planned path

Input: Moves $m_1, \ldots, m_K$ ($K \leq 100$), optional world model $\hat{f}$

$s \leftarrow$ OBSERVE(); *surprise* $\leftarrow \varnothing$;
for $i \leftarrow 1$ **to** K **do**
  if $\hat{f} \neq \varnothing$ **then** $\hat{s} \leftarrow \hat{f}(s.\text{board}, m_i)$;
  HOST.ACT(m_i); $s' \leftarrow$ OBSERVE() ; ▷ *the move is spent*
  if $\hat{f} \neq \varnothing$ ***and*** *surprise* $= \varnothing$ ***and*** DIFF$_{\neg HUD}(\hat{s}, s'.board) \neq \varnothing$ **then**
    surprise $\leftarrow (i, m_i,$ differing cells) ; ▷ *reported, not fatal*
  if *level won* ***or*** *game over* ***or*** *wall hit twice* ***or*** *(s' seen in this attempt* ***and*** *guard on)* **then**
    break;
  $s \leftarrow s'$;
return *change ledgers, surprise, s'*

der, repeats a configuration visited earlier within the same attempt. In an offline counterfactual analysis across a 13-game run from 26 October, the guard would have pruned 2,331 of 9,913 exploratory actions, though six of those were valid winning transitions, demonstrating a measurable false-positive cost. During live deployment, the guard triggered 1,814 times, prompting the agent to actively circumvent the constraint by increasing single-action `play()` invocations from 8.9

While this programmatic interface minimizes physical action expenditure, it does not reduce the computational cost of inference. With 25 concurrent games sharing a single GPU, the primary operational bottleneck shifts to the model serving loop.

4.5 Serving Inside the Envelope

Rewriting the end of the prompt destroyed prefix-cache reuse on a recurrent backbone. Our backbone mixes Gated DeltaNet linear-recurrent layers with attention (Yang, Kautz, and Hatamizadeh 2024). Its recurrent state cannot be rewound, so the server can resume only from the state saved at the end of an earlier request, and the *prefix cache*—the server's store of already-processed prompt prefixes—matches nothing past the first changed token. In the build of 19 October the board picture was sent as the last message of each request and never stored, and each game gave up its GPU slot at every turn end. Of 960 requests, 959 reused exactly the 4,416 pre-warmed system-prompt tokens and prefilled everything after them again: 10.5% of prompt tokens came from cache, and the median turn took 58 s (Figure 7).

We now enforce an *append-only* invariant: every request is the previous request plus its reply and new messages, word for word. Pictures are stored as messages once per new state and kept, reasoning stays in the history, and memory changes are appended as new lines. A game keeps its slot for 20 requests before the scheduler may hand it over, so at most about one request in 20 starts cold. Over the 8.2-hour run of 23 October, 98.4% of 1.78 B prompt tokens came from

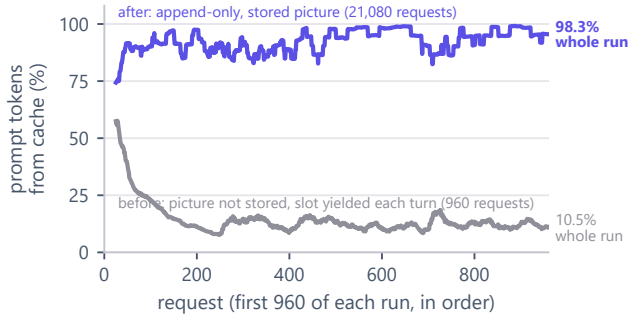


Figure 7 | Share of each request's prompt tokens served from the prefix cache (moving mean over 25 requests), for the first 960 requests of two Qwen3.8-Flash-Next runs on SGLang. Before (19 October, 15 games): the picture is the unstored last message and the GPU slot changes hands every turn. After (23 October, 25 games): every request is the previous request plus its reply, and a game keeps its slot for 20 requests. Labels on the right give the whole run.

cache and prefill took 1.3% of the busy time, against 50.1% for decoding (Table 4). The run is decode-bound.

Summarising old context costs a model call and can invent facts, so we drain it instead. Harnesses that compact context ask a model to summarise it (Knutsen and Klein 2026). The summary is model output: it costs a call on the shared GPU and can carry a fact that was never on the board into every later turn. Our *context drain* uses no model at all. When the context passes 118k tokens it drops the oldest whole turns until it is under 55k; everything that remains is word for word what the model saw, the system prompt stays pinned, and the memory and code graph are put back as one block. A drain costs one cold prefill and introduces nothing.

Working memory lives in the kernel, so it survives drains and level changes. The agent keeps four Python dictionaries in its kernel—`GOAL`, `PLAN`, `HYPOTHESES` and `FINDINGS`—keyed by level and kept across level changes. After each cleared level one curator call turns the level's confirmed and refuted hypotheses into a fifth store, `SKILLS`, and a list of refuted rules that are not proposed again (143 curator calls and 339 skills in the 23 October run). After each cell a static code graph appends one line naming the functions the cell defined or broke.

The scheduler gives the GPU to the game most likely to score next. All 25 games start at once and share 10 GPU slots. A game's priority is the RHAE weight of its current level, discounted by the actions it has already spent there, plus a small value for the levels it would unlock, times a *hope* term that halves as tokens and actions spent on the level without a win grow (Appendix C). Games that stall lose the GPU to games that progress, and a game that clears a level starts again at full priority. The constants are set by hand and not tuned.

Model and runtime. We serve Qwen3.8-Flash-Next, a 125B-parameter mixture of experts with 6B active parame-

ters per token, quantised to W4A16 with AutoRound (Cheng et al. 2023) and served by SGLang (Zheng et al. 2023) with EAGLE speculative decoding (Li et al. 2024) (3 draft tokens), an FP8 KV cache and a 136k context. Each reply is capped at 12k tokens and each game at 400 turns. A watchdog restarts a server whose token counters stop moving while requests are open, within the wall-clock budget left.

4.6 Rejected Designs

Sparse deltas. We sent only the cells that changed after each action. Actions in these games often change distant cells—stepping on a tile in `ls20` changes an icon in the status panel—and with local updates only, the agent missed the coupling and concluded its action did nothing (Section 3). Change must be reported against the whole frame, with the HUD split out, never instead of it.

Split tools. Separate tools for running code and for sending moves made the agent leave its own model to act and return to rebuild it. Merging them into `await play()` is the only recorded change between hidden-set scores of 5.14 and 5.65. An agent that plans in code must also act in code.

History stubbing. To save tokens we replaced the agent's past code in the context with a short comment. The model copied the comment as its next cell instead of writing code: in 1,855 of 2,088 calls on 16 October, and in 8,242 of 8,438 calls (97.7%) on 25 October, which cleared 0 of 97 levels. A 6B-active model imitates its own history, so anything placed where its code goes must be real code.

Subagents. The Prime Agent harness offers a call that spawns subagents. Across our first three Prime experiments and a 10-game autopsy, the open-weight models spawned none, while the port scored 0.00 to 0.27 (Section 5.1). We run one agent per game, with one curator call between levels; delegation the model never uses is complexity without benefit.

Fine-tuning on our own traces. A LoRA fine-tune on traces of cleared levels lowered the training loss from about 11% to about 6%, and the tuned model played worse; it used about 80% of our training compute. We did not find the cause. Lower imitation loss is not better play, and under a fixed compute budget the harness was the better place to spend it.

5 Results

Our empirical evaluation demonstrates that action budget exhaustion on ARC-AGI-3 is driven predominantly by perceptual misinduction rather than downstream planning failure. While deterministically computed geometric relations generalize across environments unconditionally, high-level affordance roles remain latent and require interactive disambiguation through active execution probes. To establish an uncontaminated assessment of out-of-distribution reasoning, we evaluate Sarbloh Agent primarily on the official held-out Kaggle benchmark, supplemented by hardware-matched comparative baselines and continuous systems telemetry on public environments to isolate architectural contributions from compute scale.

5.1 Hidden-Set Evaluation Trajectory

Because the official evaluation suite executes strictly offline on undisclosed game environments, it provides an uncontaminated benchmark of generalization (Figure 8). Initial baselines adapting the Prime Agent framework with open-weight models achieved scores between 0.00 and 0.27. Diagnostic trace inspection across ten games revealed that the model engaged in unguided search, initiating over 350 voluntary level restarts without resolving a single goal state.

Successive architectural interventions systematically advanced performance across five distinct milestones:

1. **Serving and Backbone Upgrade (Score 5.14)**—Migrating the serving runtime to Qwen3.8-Flash-Next on SGLang with speculative decoding, prefix-cached prompt structuring, and ten concurrent game slots established our initial operational baseline.
2. **Unified Programmatic Execution (Score 5.65)**—Merging action dispatch directly into the Python kernel via `await play()` eliminated disruptive tool-switching loops between planning and actuation (Section **??**).
3. **Deterministic Context Drainage (Score 6.06)**—Replacing lossy model-generated summaries with deterministic context truncation prevented the accumulation of hallucinated state histories (Section **??**).
4. **Symbolic Memory and AST Tracking (Score 10.98)**—Persistent level-keyed memory dictionaries, real-time code graph tracking via AST inspection, and state revisit guardrails nearly doubled the evaluation score.
5. **Integrated Multimodal Perception (Score 16.53)**—Re-enabling the full deterministic perception engine alongside refined turn-based context drainage established our peak efficiency score, representing a 13.7× improvement over the Milestone #1 baseline.

Three exploratory submissions exhibited temporary score regressions (0.35, 2.97, and 1.96, with the latter exceeding the nine-hour execution budget), which we document to illustrate the boundary conditions of runtime stability. Because the final milestone simultaneously integrated refined context drainage and the reactivation of the perception engine following intermediate ablation trials, the perception engine's exact isolated share of the final 51% gain remains coupled with these complementary runtime optimizations.

5.2 Hardware-Controlled Comparative Evaluation

To isolate architectural contributions from compute hardware, we evaluated Sarbloh directly against the Milestone #1 winning Duck harness (Tufa Labs 2026) on identical server hardware (one NVIDIA RTX PRO 6000) and identical Qwen3.8-Flash-Next model weights served via SGLang. The baseline deployed four concurrent execution passes per game, whereas Sarbloh executed a single pass across the 25 public games managed by our utility-driven priority scheduler.

As illustrated in Figure 9, Sarbloh Agent consistently accelerates solution discovery throughout the execution window, clearing 73 levels versus 54 at one hour, 97 versus 63

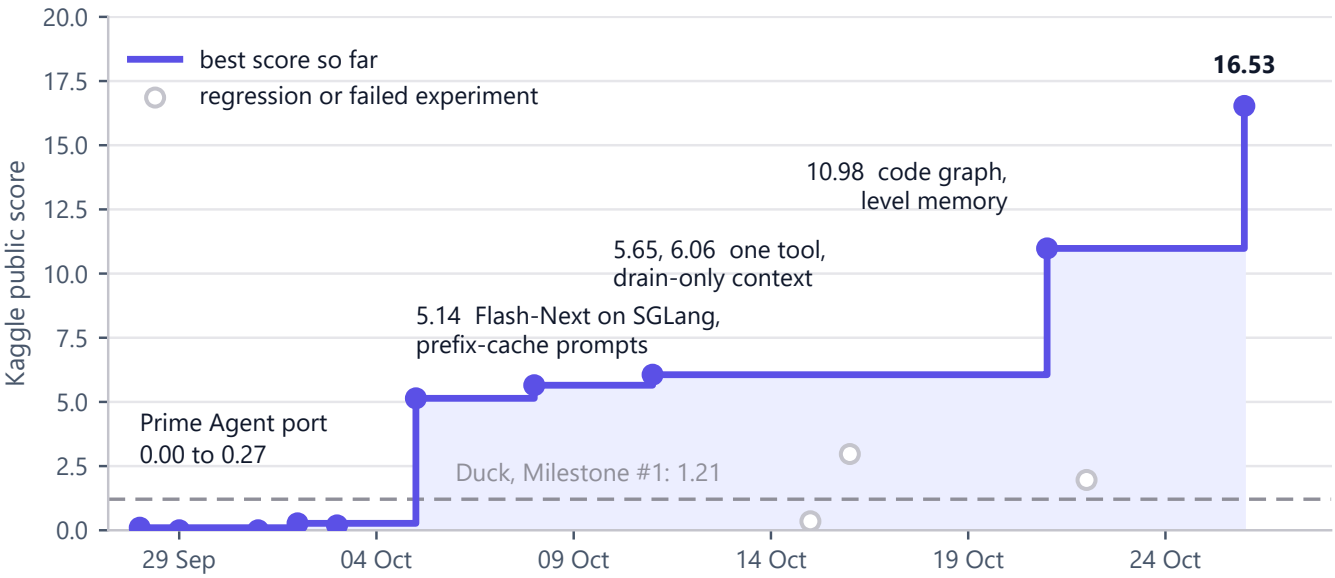


Figure 8 | Evaluation trajectory across completed submissions on the official held-out Kaggle ARC-AGI-3 benchmark. The solid curve tracks cumulative peak efficiency score, hollow markers indicate regressive exploratory trials, and annotations denote primary architectural interventions.

25 public games, one GPU	23 Oct	27 Oct
Configuration		
Briefing and map	off	on
Guardrails of 27 Oct, stuck review	–	✓
Wall clock	8.2 h	5.4 h
Outcome		
Levels cleared at 1 h / 2 h / 4 h	74 / 99 / 138	51 / 90 / 131
Levels cleared at the end	143	138
Median a_l/h_l on cleared levels	0.77	**0.68**
Mean public score	59.4	**63.5**
Prompt tokens from cache	98.4%	93.3%

Table 3 | The two full runs nearest to a perception ablation. They differ in more than the briefing, so the comparison is suggestive only. Level counts are from each run's own log of clear times.

at two hours, and 136 versus 103 at four hours. Across the complete evaluation, Sarbloh resolves 143 of 183 total levels in 8.2 hours, exceeding the 129 distinct levels completed across all four baseline passes combined. This performance advantage stems directly from dynamic priority scheduling, which concentrates GPU bandwidth on game states exhibiting high marginal progress potential rather than partitioning compute uniformly across redundant parallel trajectories.

Comparison with Frontier Model Baselines. Figure 1 compares the per-game efficiency scores of Sarbloh against five frontier models evaluated on the 25 public environments. When aggregating the best performance achieved across evaluation trials to establish an empirical upper envelope, Sarbloh achieves a mean efficiency score of 72.5, leading frontier baselines on 21 of the 25 games and achieving complete resolution on 13. Crucially, even under a single-pass execution regime on our single-GPU budget, Sarbloh achieves a mean score of 59.4, outperforming the strongest frontier model baseline of 54.8. The architecture demonstrates substantial performance advantages on complex environments such as `g50t`, scoring 54.6 while all evaluated frontier models fail to surpass 0.28. Conversely, Sarbloh trails GPT-6.1 Sol on four environments, including `dc22` (39.6 versus 47.6), `wa30` (26.3 versus 76.6), and most notably `sp80` (22.4 versus 60.1) and `tn36` (7.7 versus 75.0), where unconstrained API inference budgets favor broad exploratory search.

5.3 Where the Points Are Lost

Once a level is understood, the agent plays it in fewer actions than the human baseline. The median ratio of agent to human actions on the 143 cleared levels was 0.77, and 66% of cleared levels took no more actions than the human (Figure 10a). Under Equation (2), 62% of cleared levels earned the full 115% cap. The agent plans a level in code and sends the whole path in one call; once it knows what it is looking at, acting is not where it loses.

Levels the agent never understood cost more than twice as much as slow play on levels it did. We split the 40.6 points lost from a perfect score exactly, using the official scorer's weights (Figure 10b). Levels never reached, because the game stalled on an earlier one, cost 20.1 points; the first uncleared level of each game cost 9.2; action overhead on cleared levels cost 11.3. So 29.3 of the 40.6 points, 72%, are on levels the agent did not clear. We attribute most of these to perceptual misinduction, for two reasons visible in the traces: (i) on stuck levels the agent rewrote its stated goal a median of 8 times, against once on cleared levels; and

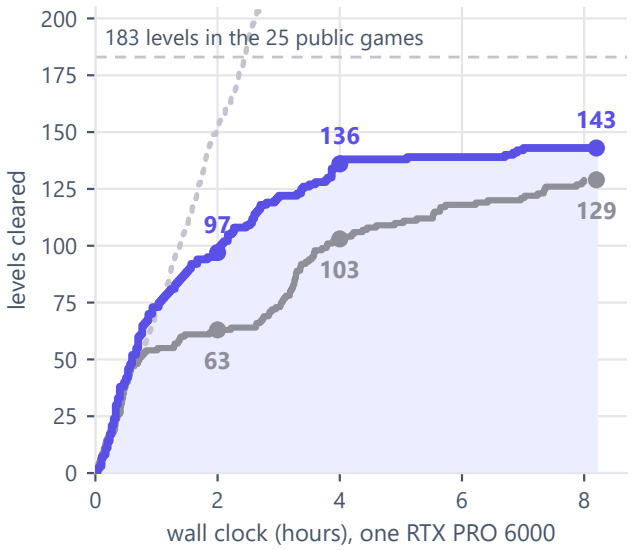


Figure 9 | Cleared levels over wall-clock time on the 25 public games, same GPU and weights. Indigo: SARBLOH, one pass. Solid grey: distinct levels cleared by the Duck harness over its four passes; dotted grey: its level count summed over the four passes. SARBLOH ends at 143 of 183 levels in 8.2 h, the Duck at 129 distinct levels.

(ii) it formed false beliefs about the game and acted on them, declaring `vc33` "unsolvable as generated" after 739 actions and then spending 1,240 more, and calling the layouts of `bp35` random. The same six games (`bp35`, `sp80`, `tn36`, `vc33`, `s5i5`, `lf52`) blocked every later level. These traces are from the briefing-off run, so they show the failure the engine is meant to remove, not the engine removing it.

5.4 Systems Telemetry

The run was decode-bound: prefill took 1.3% of busy time. Table 4 gives the telemetry of the 8.2-hour run on one RTX PRO 6000. With 98.4% of 1.78 B prompt tokens served from the prefix cache, decoding took 50.1% of busy time and prefill 1.3%. The build of 19 October, before the append-only invariant, served 10.5% from cache (Figure 7). The server answered 21,224 model calls without a failure.

5.5 Failure Modes

A low-budget prompt made four games idle for hours at their action cap. In the 23 October run, four games (`lf52`, `s5i5`, `vc33`, `re86`) stopped 2 to 15 actions short of the 2,000-action cap. The harness kept asking them to continue, and the model answered "`Holding...`" about 14,000 times, which was 1.24 B of the run's 1.78 B prompt tokens; the last level was cleared at 7.0 h and the stalls kept the run alive to 8.2 h. No game was waiting for the GPU during the stalls, so they cost wall clock, not levels. A cap of 400 turns per game now ends such games.

A placeholder in the history collapsed the agent. The two history-stubbing runs of Section 4.6 are the starkest failure we saw: with its past code replaced by a comment, the model wrote the comment as its next cell in 97.7% of calls and cleared 0 of 97 levels. Small open-weight models copy what sits in their context, and every token the harness places there is effectively an example.

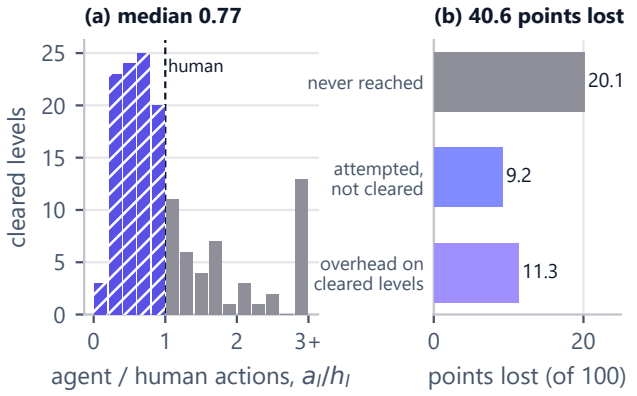


Figure 10 | The 23 October run (143 levels, mean 59.4). (a) Agent-to-human action ratio on each cleared level; hatched bars are at or below the human count. (b) The 40.6 points lost from a perfect score, computed with the official scorer's level weights: levels after the first uncleared one, the first uncleared level of each game, and the action overhead on cleared levels, net of the 115% cap.

Metric (25 games, 1 GPU, 8.2 h)	Value
Prompt tokens served from prefix cache	**98.4%**
Prompt tokens / reply tokens	1.78 B / 14.4 M
Model calls / server failures	21,224 / 0
Games inside a turn at once (mean)	11.8
Turns per minute, all games	42.7
Decode speed per request	76 tok/s
Turn time, median / 90th percentile	2.3 s / 30.2 s
Busy time: decode / prefill	50.1% / 1.3%
Reply text that is thinking	78%
Wall clock per cleared level (median)	9.2 min
a_l/h_l on cleared levels (median)	**0.77**

Table 4 | Systems telemetry of the 23 October run, from the run's speed report and server log.

6 Limitations and Future Work

Limitations. Several architectural limitations remain. First, while affordance hypotheses are coupled with unit falsification tests, the current perception engine evaluates hypotheses independently at each frame without maintaining a fully unified Bayesian belief distribution across multi-step episodes. Second, although the agent plans with super-human efficiency once a level is understood, antecedent failure on complex perceptual transitions remains the primary source of score degradation. Finally, our current results rely on general-purpose open-weight models without task-specific policy fine-tuning.

Future work. Promising research avenues include:

1. **Closed-Loop Belief Updating**: Formulating affordance induction as a dynamic partially observable belief state, where the empirical outcomes of unit falsification probes explicitly update prior belief weights across compositional levels.
2. **Offline Trajectory Fine-Tuning**: Applying parameter-efficient fine-tuning (LoRA (Hu et al. 2021)) directly on our database of verified, high-efficiency execution trajectories, optimizing syntactic code generation and world model synthesis.
3. **Sample-Efficient Reinforcement Learning**: Investigating offline reinforcement learning algorithms, such as Group Relative Policy Optimization (GRPO (Shao et al. 2024)) and balanced policy clipping, to optimize planning depth and action parsimony within our fixed computational envelope.

7 Conclusion

On an interactive benchmark with quadratic action penalties, an agent cannot pay for poor perception with trial and error. Our ablations show that language and vision-language models fail on ARC-AGI-3 mainly because unparsed token grids hide spatial and topological structure, not because they cannot reason about it.

Sarbloh addresses this with a zero-parameter, host-side perception engine that states computed geometric relations as facts and role guesses as hypotheses with one-action tests. The agent plans over these views in a persistent Python kernel and commits multi-step paths that stop when the game disagrees with a prediction. An append-only context keeps the prompt prefix-cache hit rate at 98.4% across 21,224 model calls on a single RTX PRO 6000. The agent resolves 143 of 183 public levels in 8.2 hours and scores 16.53 on the official hidden Kaggle evaluation, 13.7× the Milestone #1 benchmark.

References

ARC Prize Foundation. 2026a. ARC-AGI-3: A New Challenge for Frontier Agentic Intelligence. *arXiv preprint arXiv:2603.24621*.

ARC Prize Foundation. 2026b. ARC Prize 2026: ARC-AGI-3 Milestone #1 Results. https://arcprize.org/blog/arc-prize-2026-milestone-1.

Cheng, W.; Zhang, W.; Shen, H.; Cai, Y.; He, X.; et al. 2023. Optimize Weight Rounding via Signed Gradient Descent for the Quantization of LLMs. *arXiv preprint arXiv:2309.05516*.

Chollet, F. 2019. On the Measure of Intelligence. *arXiv preprint arXiv:1911.01547*.

Han, Q.; Hu, K.; Qiu, L.; Wu, C.; and He, K. 2026. VISTA: A Visual Harness for Reasoning in an Interactive World. https://vista-research.github.io/.

Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; et al. 2021. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*.

Karten, S.; Zhang, A. L.; Thomas, K.; Müller, S.; Bakouch, E.; et al. 2026. Prime Agent: A Self-Improving RLM Harness. *arXiv preprint arXiv:2608.23552*.

Knutsen, S.; and Klein, V. 2026. Arcgentica: ARC-AGI-3 Agent Harness Built on the Agentica SDK. https://github.com/symbolica-ai/ARC-AGI-3-Agents.

Li, Y.; Wei, F.; Zhang, C.; and Zhang, H. 2024. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. *arXiv preprint arXiv:2401.15077*.

Marr, D. 1982. *Vision: A Computational Investigation into the Human Representation and Processing of Visual Information*. W. H. Freeman.

Rahmanzadehgervi, P.; Bolton, L.; Taesiri, M. R.; and Nguyen, A. T. 2024. Vision Language Models Are Blind: Failing to Translate Detailed Visual Features into Words. *arXiv preprint arXiv:2407.06581*.

Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; et al. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*.

Spelke, E. S.; and Kinzler, K. D. 2007. Core Knowledge. *Developmental Science*, 10(1): 89–96.

Tufa Labs. 2026. The Duck: Tufa Labs' ARC-AGI-3 Harness, Milestone #1 Winner. Kaggle notebook, https://www.kaggle.com/code/jeroencottaar/tufa-labs-duck-harness-june-30-milestone-winner.

Ullman, S. 1984. Visual Routines. *Cognition*, 18(1-3): 97–159.

Wang, X.; Chen, Y.; Yuan, L.; Zhang, Y.; Li, Y.; et al. 2024. Executable Code Actions Elicit Better LLM Agents. *arXiv preprint arXiv:2402.01030*.

Yang, S.; Kautz, J.; and Hatamizadeh, A. 2024. Gated Delta Networks: Improving Mamba2 with Delta Rule. *arXiv preprint arXiv:2412.06464*.

Zheng, L.; Yin, L.; Xie, Z.; Sun, C.; Huang, J.; et al. 2023. SGLang: Efficient Execution of Structured Language Model Programs. *arXiv preprint arXiv:2312.07104*.

A Perception Lab: The Experiments Behind the Engine

This appendix records the nine perception experiments from which the engine of Section 4.2 was taken. Each experiment is a single Python file that takes one screenshot of a game frame and returns an image and a short text for a model to read. The code and all outputs are released with the paper (Appendix E).

A.1 Setup

Shared front end. All eight new experiments (X1–X8) share one front end, so that only the presentation changes. A screenshot is turned into an exact 64×64 palette-index grid (the cell pitch and origin are fitted from colour edges, and each cell takes the nearest palette colour). The grid is split into terrain (large colour fields), 4-connected objects with a kind (object, icon, dotted figure, thin line or bar), special boxes (panels) and status items at the screen edge (HUD). Every experiment then computes something on top of these objects.

Baseline. The baseline is our first perception module, the *Legacy Encoder* (`arc_eye`). It also works from a screenshot and prints a text report only, so the model never sees pixels: the fitted lattice, a palette-stable symbol per cell, terrain versus objects, connected components with holes, symmetry and a dihedral shape hash, composite objects, dotted chains and bars, and a diff with object tracking against the previous frame. It costs about 3,500 text tokens per frame.

Test frames and grading. The two test frames are the first levels of `ls20` (keyboard controls) and `r11l` (clicks). Before any experiment ran, we wrote an answer key of seven items per game and checked it against the game engine: K1 what is controlled, K2 the goal, K3 the link or body (for `ls20`, the inventory icon and the lock glyph; for `r11l`, the disc that sits at the midpoint of its two handles), K4 the modifier or hazard, K5 the status items, K6 a winning plan, and K7 the terrain. For each item, an output scored 2 if it stated or showed the item correctly and unmistakably, 1 if only partly, 0 if absent and -1 if it stated something that would mislead; the maximum is 14 per game and 28 in all. We also gave an intuition grade from 1 to 5 (how quickly a player reading only this output would understand the game) and recorded the text cost. The grader was the same agent that built the experiments and it knew both games, so it graded only what each output itself states. No blind reader was used.

A.2 The Experiments

X1 Exploded Inventory. Objects are de-duplicated by their exact colour grid. Each kind is pulled out of the scene, enlarged to a common readable size (the bounding-box cells that are empty are drawn as a checker so that holes read as holes), spaced apart, labelled with a letter, and marked $\times N$ when it has N copies. A small thumbnail of the frame outlines where each one sits, and the legend gives its zone, its ground colour and its coordinates. Thin lines are listed with their end points. The motivation is that vision-language models fail on small, close shapes but succeed when the shapes are separated (Rahmanzadehgervi et al. 2024).

X2 Surprise Map. The surprise of a cell is $-\log_2 p$, where p is the frequency of its 3×3 neighbourhood pattern in this frame. Repeated texture (floor, walls, tiles) scores near zero bits, and a pattern seen at least eight times counts as predictable. Objects are ranked by the total bits of their cells, the top eight keep full colour on an otherwise faded frame, and a symmetry-break test flags an object whose *shape* is symmetric under a mirror or quarter turn while its *colouring* is not, reporting the side that holds the odd colour.

X3 Visual Routines. Four serial routines in the sense of Ullman (1984), run in code. **Reach** picks the most self-like object (not at the screen edge, at least two colours, at least nine cells, square first). If the game has a tile of side u, it flood-fills the floor in steps of u and reports the step count to every object, or the side of a box where the floor ends. Without a tile grid it takes the open-space fraction and sweeps the smaller object's footprint along the straight line between each pair, counting rock cells hit. **Fit** lists which objects sit inside which box and which hollow figures enclose an empty space, giving its size and the objects that fit it exactly. **Trace** follows each thin line from its two end points (cells with at most one neighbour) to the nearest objects, and reports an object linked to exactly two others that lies within one cell of their midpoint. **Align** reports pairs that share at least 60% of their columns or rows.

X4 Look-alike Threads. For every pair of objects, the binary shapes are compared under the eight rotations and flips and under integer scale factors 1, 2 and 3. The relation is EXACT (a copy, which also marks counters), TURNED (equal after a rotation, flip or rescale, with the transform named), NEAR (equal size, a few cells differ, and how many), or STATE (same shape, different colour). A colour shared by three or more different things is reported as a colour link. Threads joining matched objects are drawn on the frame.

X5 Foresight Strip. The next frame is imagined for every action from four priors: an agent moves by its own size along the floor and stops at walls; the longest HUD bar is a budget that drops one unit per action; of two same-shaped objects the brighter is selected, and clicking empty space sends it there; and an object held at the midpoint of two anchors stays at their midpoint. For click games it searches symmetric anchor placements for the cheapest collision-free three-click plan. The game engine is used only afterwards to score the predictions, in a file the reader never sees.

X6 Blueprint. The frame is redrawn as a schematic after Marr (1982): floor in beige, walls and void hatched, the tile lattice drawn faintly, special boxes outlined in yellow, every object outlined and lettered with a shape word, and the HUD moved out of the map into a labelled side panel, with gauges described by their length.

X7 Role Ledger. Each object is scored against nine roles (YOU, GOAL, MODIFIER, INVENTORY, MOVES LEFT, LIVES,

Presentation	ls20 /14	r11l /14	Total /28	Int. 1–5	Tokens
Legacy Encoder (baseline)	2	1	3	1	3500
X1 Exploded Inventory	2	1	3	2	180
X2 Surprise Map	3	1	4	2	270
X3 Visual Routines	7	8	15	4	240
X4 Look-alike Threads	4	2	6	3	140
X5 Foresight Strip	4	8	12	4	140
X6 Blueprint	4	4	8	3	100
X7 Role Ledger	11	9	20	5	380
X8 Fused Briefing	13	12	25	5	370

Table 5 | Rubric score per game, intuition grade (Int.) and text tokens per frame. The Legacy Encoder also marked the move counter of r11l as screen chrome (−1). X8 was designed after the others were graded and was graded on the same two frames.

HANDLE, CARRIED THING, SCENERY) with hand-set evidence points from design priors and from the outputs of X2–X4. A long bar at the edge adds to MOVES LEFT; identical copies at the edge add to LIVES; the most self-like object adds to YOU (more if its size equals the floor tile); a hollow outline whose inside fits another object adds to GOAL; a HUD icon that matches a world object adds to INVENTORY and GOAL. A role's probability is its score divided by the object's total score plus one. Every top hypothesis is printed with its evidence and the cheapest action that would confirm or kill it.

X8 Fused Briefing. Built after X1–X7 were graded and after the held-out check of Section A.4. It uses the Blueprint canvas as its picture, draws the dashed threads of X4 and the midpoint ring of X3 on it, and writes two tiers of text: MEASURED (the facts of X3, X4 and the symmetry-break test of X2) and GUESSES (the roles of X7 with their confidence and one-action test). Objects carry the same letters in picture and text. X5 was dropped because its priors failed out of sample, and X1 and X2 on their own added too little.

A.3 Results on the Two Test Frames

Table 5 and Figure 12 give the scores and Figure 11 shows the image part of each presentation. Three things moved understanding. The *computed relations* of X3 and X4 state facts that a model cannot reliably read off a picture: on r11l X3 found the midpoint tether and the ring that fits a 5×5 object, and on ls20 X4 showed that the inventory icon is the lock glyph turned 90° and drawn at twice the size. The *role labels* of X7 gave the fastest gist, with every role right on both frames. The *foresight* of X5 predicted all four ls20 moves and the gauge, and its three-click plan solved r11l. X1 and X2 were no better than the Legacy Encoder, and X6 is the best map for navigation but says nothing about meaning. The Fused Briefing scored 25/28 at about 370 tokens per frame, 8.3× the Legacy Encoder's score with 89.4% fewer text tokens. Because the grader knew both games and X8 was graded on the frames it was designed on, these scores are optimistic.

Check on the 23 held-out games	Result
Screenshot to grid, exact cells	23/23 games
Agent identity (X7), keyboard games	6/14 games
Moves-left gauge found (X7)	10/20 games
Click handles that respond (X7)	7/29 handles
Keyboard foresight fully right (X5)	1/16 games

Table 6 | Held-out check of the single-frame guesses. For click handles, the guesses were good on dc22 (2/2), sb26 (4/4) and ka59 (1/1) and bad on lp85 (0/17). On two further keyboard games nothing moved on the first key press, so the agent-identity guess could not be checked.

A.4 Held-Out Check: Facts Versus Guesses

We then checked X5 and X7 on the 23 other official games, none of which was used in the design. The checker synthesises a screenshot from each game's reset frame (7 pixels per cell) and sends it through the normal loader, and ground truth comes from the game engine only: the agent is the objects whose cells change under the arrow actions, an object responds to clicks if clicking its centre changes any non-HUD cell, and the gauge is right if the bar the ledger calls MOVES LEFT changes after one action (Table 6). Turning a screenshot into a grid generalised perfectly. The guesses did not: they had effectively been fitted to the two test frames, and were right about half the time or less on unseen games.

This fixed the design rule of the engine: measured relations (X3, X4) are true by construction and carry no such risk, so they are stated as facts, whereas roles and dynamics from one frame are stated as guesses with a one-action test. We did not run the Fused Briefing as a whole on the held-out games, and the lab reports no RHAE. The end-to-end effect of the engine on the agent is not isolated by an ablation: the trajectory of Section 5.1 changes several components at once.

B Perception Engine Structural Briefings

Below we provide the verbatim structured briefing text emitted by the deterministic perception engine for the opening frames of ls20 and r11l illustrated in Figure 5. The object identifiers correspond exactly with the annotated spatial map and the kernel's symbolic state dictionary (observe().objects).

```
BRIEFING (arrow keys)
MEASURED (certain):
- Floor is a 5x5 tile grid.
- obj 4 (blue 6-cell shape): sits inside a black box
    that is NOT plain floor; the floor reaches the box's
     bottom side in 5 steps (the only way in)
- obj 6 (white/light grey plus): reachable in 6 steps
- obj 4 (blue 6-cell shape) is directly above obj 10 (
    blue/orange square) (shared columns 35-37)
- obj 12 (blue irregular shape (24 cells)) (drawn 2x
    bigger), mirrored left-right OR turned 90 deg
    clockwise, becomes exactly obj 4 (blue 6-cell shape)
```

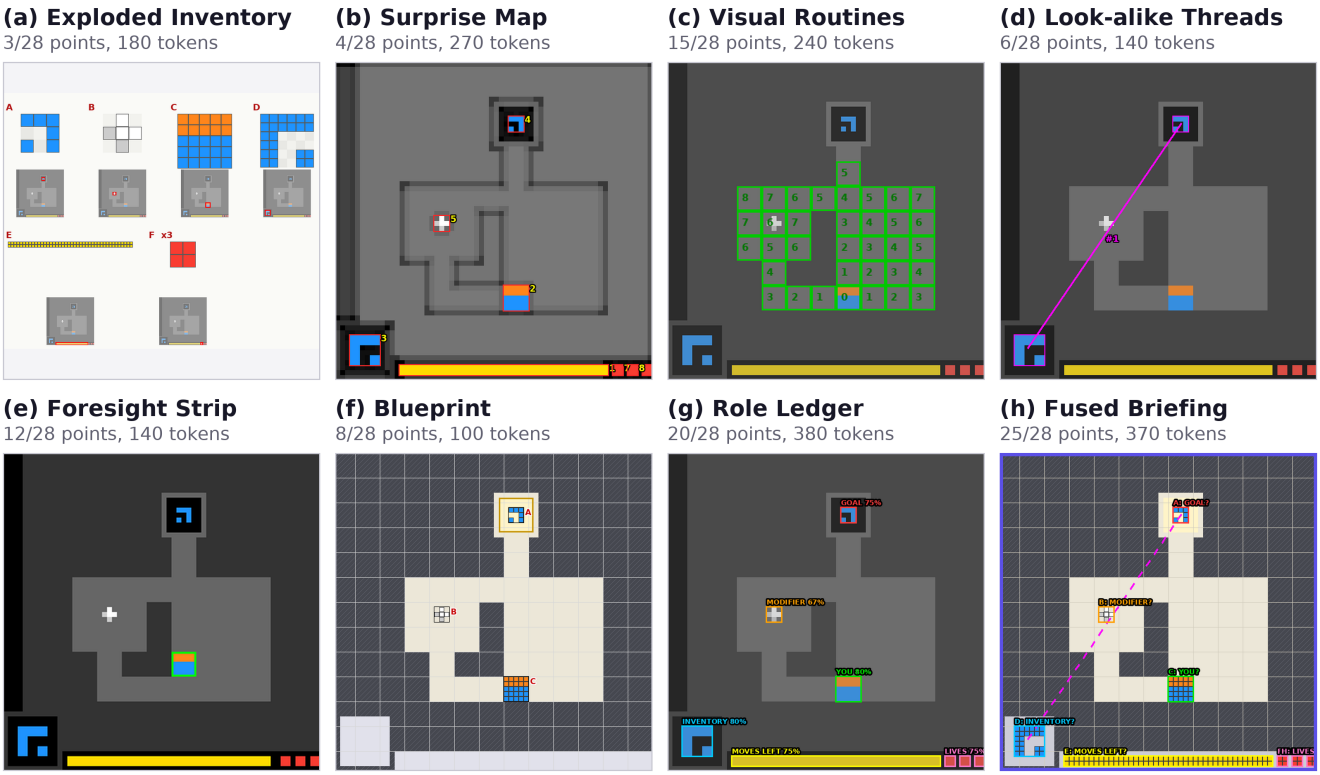


Figure 11 | The image part of each candidate presentation for the opening frame of `ls20`, with its rubric score and its text cost per frame. (a) Exploded Inventory pulls out every object and stacks copies. (b) Surprise Map keeps colour only where the frame is unpredictable. (c) Visual Routines marks reachability, with the step count to each tile. (d) Look-alike Threads joins shapes that match under rotation, mirroring or scale. (e) Foresight Strip imagines the next frame (here, one step up). (f) Blueprint redraws floor, walls and lattice as a clean schematic. (g) Role Ledger tags each object with a guessed role and a confidence. (h) Fused Briefing, the presentation we kept, combines the blueprint with measured facts and tagged guesses.

```
- 3 identical red square (2x2): obj 16, 17, 18 -> a
    count of something (lives, items, tiles)
- obj 6 (white/light grey plus) has a symmetric shape
    but lopsided colours (marks a direction/state)
- obj 10 (blue/orange square) has a symmetric shape but
    lopsided colours (marks a direction/state)
GUESSES (check with one action):
- obj 10 (blue/orange square) = YOU (80%) - test: press
    one arrow key: it should move one tile and the gauge
     should drop
- obj 12 (blue irregular shape (24 cells)) = INVENTORY
    (80%) - test: watch it after touching the modifier
- obj 4 (blue 6-cell shape) = GOAL (75%) - test: get the
     right thing into it and see if the level ends
- obj 15 (yellow bar) = MOVES LEFT (75%) - test: take
    any action and check it shrinks by one unit
- obj 16, 17, 18 (red square) = LIVES (75%) - test:
    watch it when the gauge runs out or you fail
- obj 6 (white/light grey plus) = MODIFIER (67%) - test:
     step onto it, then check whether the inventory icon
     changed


BRIEFING (clicks)
MEASURED (certain):
- no tile grid -> free movement. Open space = black (81%
     of screen).
- all 4 main objects share one open space; straight
    paths between them are ALL CLEAR of rock
- the grey areas are not open floor (likely walls)
- obj 4 (purple dotted ring) encloses an EMPTY 5x5 space
     -- exactly the size of: obj 16 (white/purple
    diamond), obj 25 (purple/magenta disc), obj 36 (dark
     grey/purple diamond)
- a light grey line connects obj 16 (white/purple
    diamond) to obj 25 (purple/magenta disc)
- a light grey line connects obj 25 (purple/magenta disc
    ) to obj 36 (dark grey/purple diamond)
- obj 25 (purple/magenta disc) sits at the MIDPOINT of
    obj 16 (white/purple diamond) and obj 36 (dark grey/
    purple diamond) (error 0.5 cells) -> it is held
    between them; moving either end should move it to
    the new midpoint
- obj 16, 36 (white/purple diamond; dark grey/purple
    diamond) have the same shape in different colours (
    white/purple vs dark grey/purple)
GUESSES (check with one action):
- obj 4 (purple dotted ring) = GOAL (80%) - test: get
    the right thing into it and see if the level ends
- obj 16, 36 (white/purple diamond; dark grey/purple
```

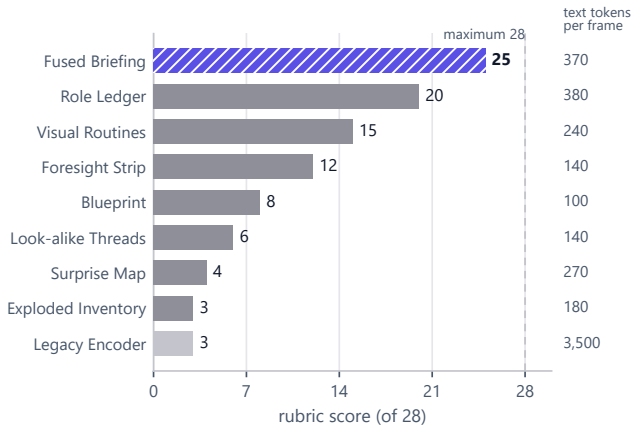


Figure 12 | Rubric score (of 28) and text tokens per frame for the Legacy Encoder and the eight presentations, on the two test frames. The Fused Briefing scores 25/28 against 3/28 for the Legacy Encoder at about a tenth of the text.

```
    diamond) = HANDLE (click) (80%) - test: click an
    empty spot once: the selected handle should jump
    there
- obj 1 (white bar) = MOVES LEFT (75%) - test: take any
    action and check it shrinks by one unit
- obj 25 (purple/magenta disc) = CARRIED THING (75%) -
    test: move one handle: it should slide to the new
    midpoint
```

C System Overview and Scheduler

Figure 13 shows how the components connect. The scheduler of Section 4.5 gives a game playing level $L{+}1$ of n, having spent t reply tokens and a actions on that level without a win, the priority

$$\pi = \big(g_{\text{now}} + g_{\text{tail}}\big) \cdot \underbrace{2^{-\left((t/T)^2+(a/A)^2\right)}}_{\text{hope}}, \tag{3}$$

$$g_{\text{now}} = \frac{L+1}{n(n+1)/2}\left(\frac{h}{h+a}\right)^2, \quad g_{\text{tail}} = \frac{\tau_{\text{tail}}(n-L-1)}{55}\,\phi,$$

where g_{now} is the RHAE weight of the current level discounted by the actions already spent, g_{tail} the value of the levels it unlocks, with $h = 25$ actions (a typical human level), $T = 40$k tokens and $A = 115$ actions. The tail value $\tau_{\text{tail}}(k)$ is 8, 7, 5 or 0 when $k \geq 3$, 2, 1 or 0 levels remain after the current one, on the 55-weight scale of a 10-level game, so that short games do not starve long ones; $\phi \in [0, 1]$ fades it to zero over the last 20% of the run budget, when no time is left for later levels. A game that has never held a slot goes before every game that has. A game keeps a slot for 20 model calls before it can be handed over. All constants were set by hand; we did not tune them.

D System Configuration and Hyperparameters

Table 7 documents the primary execution parameters governing the scored benchmark evaluation.

System Configuration Parameter	Operational Value
Concurrent games / Active GPU slots	25 games / 10 slots
Scheduler quantum allocation	20 model calls per round
Action allocation ceiling per game	2,000 actions
Maximal steps per `play()` path	$\leq$100 actions
Turn reply budget / Inner requests	12k tokens / 12 calls
Turn preemption thinking threshold	16k tokens
Context drain threshold / target	118k tokens / 55k tokens
Context maximum window size	131,072 tokens
Visual map rendering scale	10 pixels per grid cell
Briefing line limit / Short-view ceiling	20 lines / 600 tokens
Symbolic memory caps (G/P/H/F)	150 / 200 / 400 / 300 tokens
Sampling temperature / top-p / top-k	$T = 0.6$ / $p = 0.95$ / $k = 20$
Serving engine and precision	SGLang, W4A16, FP8 KV cache
Speculative decoding configuration	EAGLE (3 speculative draft steps)

Table 7 | System hyperparameter settings deployed in the official evaluation runtime.

E Artifacts and Reproducibility

All empirical findings, telemetry traces, and figures in this manuscript are fully reproducible using the scripts in `Writing/scripts/`:

- `collect_data.py`: Ingests the Kaggle submission log, per-game benchmark tables, and perception lab evaluation rubrics.
- `extract_curves.py`: Extracts wall-clock level completion trajectories from raw execution logs.
- `make_perception_figure.py`: Executes the host-side perception engine on raw game environments to generate segmented objects, spatial maps, and structured briefings.
- `plot_results.py`: Generates all publication-grade figures in vector PDF and high-resolution raster formats.

All code, model serving scripts, evaluation environments, and experimental logs are publicly accessible under the MIT-0 license (a permissive public-domain-equivalent license, as the competition rules require) at https://github.com/Harsidak/SARBLOH.

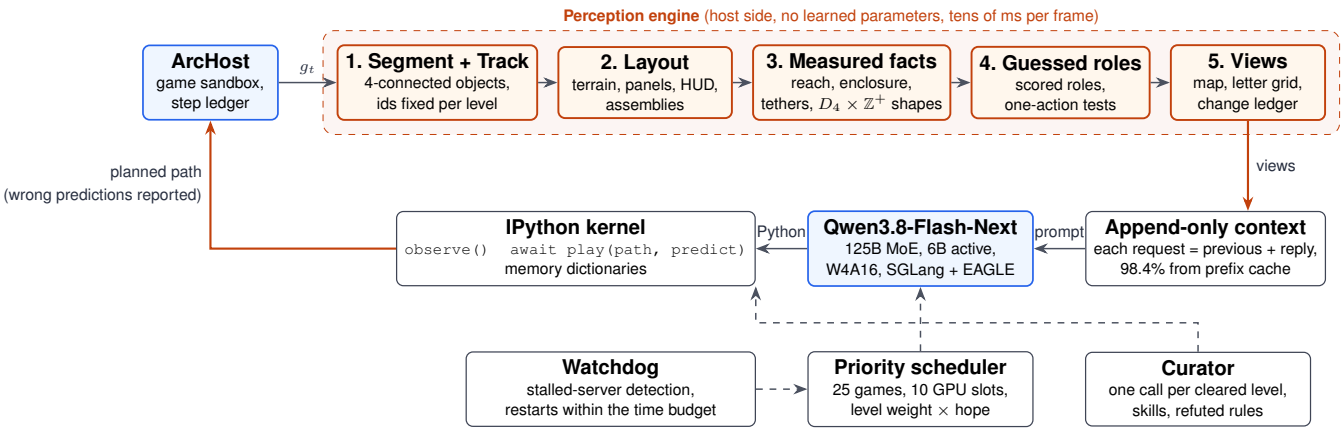


Figure 13 | Components of SARBLOH. The perception engine (orange) turns each frame into the three views of Section 4.2; the model acts only through Python in its kernel; 25 games share one GPU under the scheduler.